

LATIHAN

Studi Kasus Utama: Sistem Manajemen Rumah Sakit Sederhana

Bagian 1:

Sistem ini membutuhkan pencatatan entitas mandiri yang terlibat di rumah sakit, yaitu tenaga medis (dokter) dan pengunjung (pasien). Sistem harus mencatat identitas dan spesialisasi dokter, serta mendata identitas dan umur pasien. Dalam operasional harian, sistem harus mampu mensimulasikan interaksi ketika seorang dokter memeriksa seorang pasien. Aktivitas ini harus memunculkan rekam jejak (cetak ke layar) yang memperlihatkan nama dan spesialisasi dokter yang bertugas, beserta nama dan umur pasien yang sedang ditanganinya. Secara logis, dokter dan pasien memiliki kehidupannya masing-masing dan tidak ada kepemilikan mutlak di antara keduanya; mereka hanya terhubung pada saat proses pemeriksaan terjadi. Buatlah *class* untuk entitas-entitas ini beserta metode interaksinya.

Bagian 2

RS Sehat Selalu secara fisik terdiri dari beberapa ruangan operasional. Setiap ruangan memiliki nomor registrasi (misalnya "R-01") dan kapasitas maksimal pasien. Sebagai sebuah infrastruktur, ruangan-ruangan ini adalah bagian fisik yang tidak terpisahkan dari gedung rumah sakit utama. Manajemen mensyaratkan bahwa apabila entitas rumah sakit tersebut ditiadakan atau dihapus dari sistem, maka seluruh entitas ruangan yang ada di dalamnya harus otomatis ikut musnah dan tidak boleh eksis lagi secara mandiri. Rancanglah *class* utama untuk Rumah Sakit yang secara internal membangun dan menyimpan infrastruktur ruangnya sendiri sesaat setelah rumah sakit tersebut didirikan (beroperasi). Untuk ini, batasi kapasitas sistem agar hanya menampung 2 ruangan menggunakan *array* biasa. Sediakan pula kemampuan agar rumah sakit dapat mencetak daftar ruangan yang dimilikinya.

Bagian 3

Selain infrastruktur fisik, RS Sehat Selalu juga mempekerjakan sekumpulan tenaga medis. Sistem rumah sakit harus memiliki wadah (menggunakan *array* biasa) untuk menampung daftar dokter yang ditugaskan di sana. Berbeda dengan ruangan, para tenaga medis ini berstatus sebagai kontraktor independen yang datanya dikelola oleh sistem pusat. Hal ini berarti, para dokter tersebut sudah ada sebelum ditugaskan ke RS Sehat Selalu. Dan yang terpenting, jika RS Sehat Selalu suatu saat gulung tikar dan entitasnya dihancurkan dari memori sistem, data para dokter tersebut tidak boleh ikut musnah karena mereka harus bisa ditarik kembali dan ditugaskan ke rumah sakit cabang lainnya. Implementasikan rancangan ini dan sediakan fitur agar rumah sakit dapat menampilkan daftar dokter yang sedang bertugas di tempat tersebut.

Bagian 4

Rangkai seluruh arsitektur yang telah Anda buat ke dalam sebuah *Main Class*. Anda harus mendemonstrasikan penciptaan setidaknya 2 dokter dan 2 pasien dari sistem pusat. Tunjukkan bagaimana salah satu dokter memeriksa pasien. Selanjutnya, dirikan "RS Sehat Selalu", masukkan para dokter tersebut ke dalam rumah sakit, dan tampilkan seluruh data ruangan beserta data dokter yang ada di sana. Sebagai pengujian terakhir, objek "RS Sehat Selalu" (jadikan *null*). Tuliskan pembuktian Anda di akhir program (melalui baris komentar dan eksperimen pemanggilan objek) untuk menjelaskan entitas apa saja yang masih ada di memori dan entitas apa saja yang dipastikan hilang.

POST-TEST

Studi Kasus Lanjutan: Sistem Simulasi Perakitan di Pabrik Otomotif

Sebuah pabrik otomotif sedang membuat sistem simulasi perakitan dan pengecekan mobil. Dalam sistem ini, terdapat entitas independen berupa Ban yang diproduksi terpisah di pabrik komponen (dengan atribut merk dan ukuranRing). Karena bersifat mandiri, ban-ban ini harus tetap ada dan bisa disimpan kembali di gudang jika mobil utamanya dihancurkan. Di sisi lain, setiap entitas Mobil (dengan atribut merkMobil dan warna) memiliki komponen internal berupa Mesin (atribut nomorSeri dan kapasitasCC) yang tidak dirakit terpisah, melainkan harus diinstansiasi secara langsung tepat saat kerangka mobil dibangun di dalam *constructor*-nya. Jika mobil dilebur, mesin tersebut harus otomatis musnah bersamanya. Selain itu, pabrik mempekerjakan Montir (dengan idMontir dan nama) yang bertugas memeriksa kelayakan mobil melalui metode lakukanQualityControl(Mobil m), di mana keduanya adalah entitas mandiri yang hanya berinteraksi sesaat. Rancanglah semua kelas tersebut! Pastikan kelas Mobil memiliki metode pasangSetBan(Ban[] setBan) untuk memasukkan maksimal 4 buah ban ke dalam sebuah *array*, serta metode tampilkanSpesifikasi() untuk mencetak seluruh detail komponennya. Terakhir, buatlah *Main Class* untuk mensimulasikan penciptaan 4 ban mandiri, pembentukan 1 unit mobil, pemasangan ban ke mobil, serta proses inspeksi oleh 1 orang montir. Akhiri simulasi dengan skenario mobil "gagal uji kelayakan" sehingga harus dihancurkan (di-set *null*), lalu buktikan melalui kode dan komentar bahwa entitas ban masih tetap selamat di memori sedangkan entitas mesin telah musnah tanpa jejak!

TAKE HOME TASK

Sebuah bank digital bernama "NeoBank" menugaskan Anda untuk merancang purwarupa arsitektur inti dari sistem perbankan mereka. Sebagai fondasi keamanan dan standarisasi sistem, Anda diwajibkan untuk mengimplementasikan prinsip abstraksi dengan membuat sebuah *interface* bernama Otorisasi yang mensyaratkan adanya metode verifikasiPIN(). Selanjutnya, buatlah sebuah *abstract class* bernama Rekening yang secara struktural mengimplementasikan *interface* tersebut. Kelas abstrak ini bertugas menyimpan data krusial seperti nomor rekening, nama pemilik, dan saldo rekening. Terapkan prinsip enkapsulasi secara ketat di sini; saldo tidak boleh diakses atau diubah secara langsung dari luar kelas, melainkan hanya dapat bertambah atau berkurang melalui metode setor() dan tarik(). Dari kelas abstrak ini, Anda harus menurunkan dua entitas konkret melalui konsep pewarisan, yaitu RekeningReguler dan RekeningPrioritas. Terapkan polimorfisme dengan melakukan *overriding* pada metode tarik(); pemegang akun reguler akan selalu dikenakan potongan biaya admin tetap untuk setiap penarikan, sedangkan akun prioritas sepenuhnya dibebaskan dari biaya admin namun memiliki syarat batas minimum penarikan yang jauh lebih ketat yang harus divalidasi oleh sistem.

Selain hierarki struktural di atas, arsitektur perbankan ini sangat sarat dengan relasi antar objek. Terapkan konsep komposisi pada entitas Rekening dengan memastikan bahwa setiap kali sebuah entitas rekening diterbitkan, sistem harus secara otomatis menginstansiasi sebuah objek BukuMutasi secara internal tepat di dalam *constructor*-nya. Objek mutasi ini bertugas mencatat log aktivitas (cukup mencetak histori via *print*) dan secara logis objek mutasi ini harus ikut musnah tanpa sisa jika suatu saat rekening tersebut ditutup atau dihancurkan oleh sistem pusat. Di sisi lain, sistem juga harus mengelola entitas Nasabah. Terapkan prinsip agregasi di mana seorang nasabah memiliki kapasitas untuk menampung maksimal tiga buah objek Rekening di dalam profil mereka menggunakan struktur *array*. Rekening-rekening ini bersifat *loose-coupling*; mereka adalah entitas mandiri yang bisa saja dipindahtangankan atau tetap tercatat di bank data pusat (tidak hancur) meskipun profil nasabah yang bersangkutan sedang dinonaktifkan atau dihapus dari memori. Untuk melengkapi alur operasional, wujudkan bentuk asosiasi umum di mana

entitas Nasabah dapat berinteraksi dengan entitas CustomerService melalui metode pelaporan keluhan, di mana kedua objek ini berdiri sendiri tanpa ada unsur kepemilikan satu sama lain.

Rakit seluruh arsitektur *backend* yang telah Anda buat ke dalam kelas Main yang berfungsi sebagai antarmuka pengguna di terminal. Gunakan Scanner dan perulangan while atau do-while untuk menyajikan menu sistem perbankan interaktif. Melalui menu ini, pengguna harus mampu mendaftarkan profil nasabah, membuka jenis rekening pilihan mereka untuk dimasukkan ke profil, melakukan simulasi login, melakukan transaksi penyetoran dan penarikan yang memicu metode mutasi, serta menghubungi *customer service*. Di akhir kode program Anda harus menyediakan skenario penutupan akun secara paksa (men-set profil nasabah menjadi *null*) beserta baris komentar analitis yang membuktikan bahwa aturan siklus hidup objek pada komposisi dan agregasi yang Anda buat telah berjalan sesuai dengan landasan teori Pemrograman Berorientasi Objek.